[Trung Nguyen]Interprocess Communication - IPC

Interprocess Communication (IPC)

Table of Contents

- I. IPC Overview
- II. Pipe
- III. FIFO (Named Pipe)
- IV. Shared Memory
- V. Signal

I. IPC Overview

1. Process isolation

A **process** is a program in execution, given its own resources by the kernel. Each process owns a private **virtual address space**; the kernel maps these virtual addresses onto physical memory. The same virtual address in two different processes may map to two different regions of physical memory.

Process isolation is the property that a process cannot directly access another process's memory. It provides memory protection and limits the blast radius of a faulty process, but it also

means processes cannot exchange data through shared variables.

Threads within one process share a single address space, so they access shared data directly and only need synchronization primitives such as a mutex, condition variable, semaphore or rwlock.

2. Definition of IPC

Interprocess Communication (IPC) is the set of kernel-provided mechanisms that let independent processes exchange data and coordinate execution while keeping separate address spaces. IPC is not a single mechanism but a family of mechanisms with different properties.

3. Two basic needs

Communication is the transfer of information from one process to another. The information may be data, a command, a request, a response, an event or a status.

Synchronization is the coordination of the timing and ordering of execution between processes, so that access to a shared resource is safe.

Two related concepts:

- **Race condition**: a situation where the result depends on the order or timing in which processes or threads run.
- **Critical section**: a region of code that accesses a shared resource and must be protected so that only one party executes it at a time.

4. Classification of IPC mechanisms

IPC mechanisms fall into three main groups.

Communication covers three kinds:

- **Byte stream**: transfers a continuous stream of bytes with no boundaries between individual sends. This includes pipes, FIFOs and stream sockets.
- **Message**: transfers discrete messages that the receiver reads whole. This includes message queues (System V and POSIX) and datagram sockets.
- **Shared memory**: several processes map the same physical memory region; no copying through the kernel is needed, but the processes must synchronize access themselves.

Synchronization covers semaphores, file locks, and mutexes or condition variables configured as process-shared.

Signal is an asynchronous notification sent by the kernel or by another process to report that an event has occurred. A signal carries very little application data — essentially just the signal number — so it suits event notification and simple synchronization but not data transfer.

II. Pipe

1. Definition

A **pipe** (anonymous pipe) is a unidirectional, byte-stream channel managed by the kernel, created with the `pipe()` system call.

```
#include <unistd.h>
int pipe(int pipefd[2]);   /* pipefd[0]: read end, pipefd[1]: write end */
```

A **file descriptor (fd)** is an integer a process uses to refer to an I/O resource managed by the kernel. `pipe()` returns two of them: one for the read end, one for the write end.

`fork()` creates a child process that is a copy of the parent at the moment of the call. The child inherits all of the parent's open file descriptors, which is how both ends up referring to the same pipe.

2. Properties

No name. A pipe does not exist in the filesystem and is reachable only through inherited file descriptors. A pipe therefore works only between related processes (parent–child, sibling).

Unidirectional. A pipe has one read end and one write end. Bidirectional communication requires two separate pipes.

Byte stream. A pipe does not record boundaries between `write()` calls; it only guarantees that byte order is preserved. An application that needs discrete messages must frame them itself.

Finite capacity. The pipe's kernel buffer has a limited size. When the buffer is empty the reader blocks; when it is full the writer blocks. **Blocking** is the state in which the kernel suspends a process until a condition is satisfied, consuming no CPU.

Lifetime tied to file descriptors. A pipe is destroyed when every file descriptor referring to it has been closed. It leaves nothing behind and needs no cleanup.

3. Read and write semantics

For `read()`:

- If the pipe holds data, `read()` returns immediately with at most the requested number of bytes.
- If the pipe is empty and at least one write fd is still open, `read()` blocks.
- If the pipe is empty and every write fd is closed, `read()` returns `0`. This value is **EOF (End Of File)** and means no more data will ever arrive.

For `write()`:

- If the pipe has room, the data is written and `write()` returns.
- If the pipe is full, `write()` blocks; if the fd is in `O_NONBLOCK` mode, `write()` returns `-1` with `errno = EAGAIN`.
- If every read fd is closed, the kernel sends `SIGPIPE` to the writing process; if that signal is ignored, `write()` returns `-1` with `errno = EPIPE`.

4. When a pipe is appropriate

A pipe fits when the processes are related, the data flows in one direction as a continuous byte stream, and the processes share a common lifetime. When any of these does not hold, consider a FIFO, a socket, a message queue or shared memory.

---

5. Common problems

5.1. `read()` blocks forever because a write fd was left open

Symptom. The reading process consumes all the data, but the next `read()` never returns. The parent waiting in `wait()` is stuck as well, producing a deadlock.

Code that causes it.

```
/* CHILD: does not close the write end inherited from the parent */
/* close(pipefd[1]); */
n = read(pipefd[0], buf, sizeof(buf));   /* first call: returns 3 */
n = read(pipefd[0], buf, sizeof(buf));   /* second call: blocks forever */
```

Cause. After `fork()`, both parent and child hold a write fd. The kernel reports EOF only when the total number of open write fds reaches zero, and it does not care which process holds them. A reader that still holds its own write fd makes the kernel assume more data may arrive, so it keeps

blocking. The bug only shows up when the pipe is empty, because `read()` checks the write-fd count only when there is no data.

Fix. Immediately after `fork()`, each process closes the end it does not use.

```
if (pid == 0) {          /* child: reads only */
    close(pipefd[1]);
} else {                 /* parent: writes only */
    close(pipefd[0]);
}
```

5.2. Loss of message boundaries

Symptom. Three writes of `"AAA"`, `"BBB"` and `"CCC"` may be received by a single `read()` as `"AAABBBCCC"`. The result varies between runs, and with a small read size a message can be cut in half.

Code that causes it.

```
/* PARENT: three writes */
write(pipefd[1], "AAA", 3);
write(pipefd[1], "BBB", 3);
write(pipefd[1], "CCC", 3);

/* CHILD: one read, n = 9, buf = "AAABBBCCC" */
n = read(pipefd[0], buf, sizeof(buf) - 1);
```

Cause. A pipe is implemented as a queue of bytes and stores no length for individual writes. `read()` returns as many bytes as are currently buffered, regardless of how many `write()` calls produced them. The amount returned by each read depends on when the scheduler switches between the processes.

Fix. The application must frame messages itself (**message framing**), either with a **delimiter** such as `'\n'` or with a fixed-size **length prefix** ahead of each message. Always check the return value and read in a loop until the required number of bytes has arrived.

```
ssize_t read_full(int fd, void *buf, size_t len) {
    size_t total = 0;
    while (total < len) {
        ssize_t n = read(fd, (char *)buf + total, len - total);
        if (n == 0) break;                  /* EOF */
        if (n == -1) {
```

```
            if (errno == EINTR) continue;  /* interrupted by a signal */
            return -1;
        }
        total += n;
    }
    return total;
}
```

`EINTR` is the error code reporting that a system call was interrupted by a signal before completing; the call must be reissued.

5.3. The process is killed by `SIGPIPE`

Symptom. The process dies at `write()` with no error message and none of the statements after `write()` run. The shell reports exit status `141`.

Code that causes it.

```
close(pipefd[0]);            /* parent closes the read end */
/* the child closed its read end and exited, so no read fd remains */

write(pipefd[1], "hello", 5);   /* kernel raises SIGPIPE; never returns */
printf("still alive\n");        /* never runs */
```

Cause. Writing to a pipe with no open read fd makes the kernel send the **`SIGPIPE`** signal (number 13) to the writer. The default action for `SIGPIPE` is to terminate the process, so `write()` never returns. The shell reports the exit status of a signal-terminated process as 128 plus the signal number, hence 141.

This default exists for shell pipelines: when the reader exits (for example `head` in `yes | head -3`), the writer terminates instead of running forever.

Risk. `SIGPIPE` acts on the whole process. In a server handling many connections, one peer disconnecting can take the entire server down.

Fix. Ignore `SIGPIPE` and handle the `EPIPE` error per file descriptor.

```
signal(SIGPIPE, SIG_IGN);

if (write(fd, buf, len) == -1 && errno == EPIPE) {
    close(fd);   /* close only the broken channel */
}
```

5.4. Finite pipe capacity

Symptom. Writing continuously with no reader stops after 65536 bytes on Linux.

Code that causes it.

```
/* no fork, so there is no reader */
fcntl(pipefd[1], F_SETFL, fcntl(pipefd[1], F_GETFL) | O_NONBLOCK);

char block[1024];
long total = 0;
while (write(pipefd[1], block, sizeof(block)) > 0)
    total += 1024;           /* stops at 65536 with errno = EAGAIN */
```

Cause. The pipe's kernel buffer has a limited size. That size is an implementation detail rather than a standard guarantee: on Linux since 2.6.11 the default is 65536 bytes. The file `/proc/sys/fs/pipe-max-size` sets the maximum an unprivileged user may request, not the default capacity. On Linux the capacity of a particular pipe can be queried with `fcntl(fd, F_GETPIPE_SZ)` and changed with `fcntl(fd, F_SETPIPE_SZ, size)`.

Measuring the capacity. To measure without blocking, the write end is switched to non-blocking mode. **`O_NONBLOCK`** is an fd status flag; when set, an operation that would block returns `-1` with `errno = EAGAIN` instead. The flag does not change the capacity, only the reaction when the limit is reached.

```
int flags = fcntl(fd, F_GETFL);           /* read the current flags */
fcntl(fd, F_SETFL, flags | O_NONBLOCK);   /* add the flag, keep the rest */
```

Reading the flags before adding one is mandatory; calling `fcntl(fd, F_SETFL, O_NONBLOCK)` directly clears the other status flags.

Fix. A program must not depend on a specific capacity value; it must write in a loop and check the return value (see 5.5).

5.5. `write()` blocks when the pipe is full

Symptom. The writer runs fast, then stalls at one `write()` until the reader starts consuming. `write()` may also return fewer bytes than requested.

Code that causes it.

```
/* PARENT: writes continuously */
for (int i = 0; i < 100; i++) {
    printf("about to write #%d\n", i);
    fflush(stdout);
    write(pipefd[1], block, 4096);   /* blocks here once the pipe is full */
}

/* CHILD: reads slowly */
sleep(20);
```

Cause. When the pipe is full, the kernel puts the writer to sleep and wakes it once buffer space becomes available. This is **flow control**: the writer's rate is automatically limited to the reader's rate, preventing unbounded buffer growth. A **partial write** occurs when the buffer has room for only part of the data.

Debugging note. `stdout` is buffered by the C library, so call `fflush(stdout)` before an operation that may block to make the output appear at the right time. The last log line printed before `write()` corresponds to the write that is currently blocked, not to the last successful write.

Fix. Write in a loop until all the data has been transferred.

```
ssize_t write_full(int fd, const void *buf, size_t len) {
    size_t total = 0;
    while (total < len) {
        ssize_t n = write(fd, (const char *)buf + total, len - total);
        if (n == -1) {
            if (errno == EINTR) continue;
            return -1;
        }
        total += n;
    }
    return total;
}
```

5.6. Interleaved data when several processes write

Symptom. With two processes writing to one pipe, writes of at most 4096 bytes always arrive contiguously. Writes larger than 4096 bytes get interleaved with the other process's data inside a single write, and the degree of interleaving varies between runs.

Code that causes it.

```
/* two child processes write to one pipe; size comes from the command line */
memset(buf, 'A' + i, size);          /* child 0 writes 'A', child 1 writes 'B' */
for (int k = 0; k < 20; k++)
    write(pipefd[1], buf, size);     /* size = 4096 contiguous, 8000 interleaved */
```

Cause. POSIX defines the constant **`PIPE_BUF`** (declared in `<limits.h>`) as the threshold that guarantees an **atomic** `write()`. An atomic operation either completes in full or does not happen at all, and cannot be interrupted by another operation.

- For a `write()` of at most `PIPE_BUF` bytes, the data is written whole and is never interleaved with another process's `write()`.
- For a `write()` larger than `PIPE_BUF`, the kernel guarantees nothing: the data may be written in parts as the buffer fills, and another process can write in between while the first waits.
- In `O_NONBLOCK` mode, a `write()` of at most `PIPE_BUF` bytes that does not fit fails with `EAGAIN` instead of writing part of the data.

On Linux `PIPE_BUF` is 4096. POSIX only requires a minimum of 512 (`_POSIX_PIPE_BUF`).

Risk. The bug is hard to catch because test data is usually small and stays below the threshold; interleaving appears only once real data exceeds it.

Fix. Write each message with exactly one `write()` call of at most `PIPE_BUF` bytes. Atomicity applies to an individual call, never to a sequence of calls. Portable code must not assume `PIPE_BUF` is 4096.

```
/* WRONG: three independent calls, another process can write in between */
write(fd, "[LOG] ", 6);
write(fd, text, len);
write(fd, "\n", 1);

/* RIGHT: a single call, at most PIPE_BUF bytes */
char buf[512];
int n = snprintf(buf, sizeof(buf), "[LOG] %s\n", text);
write(fd, buf, n);
```

III. FIFO (Named Pipe)

1. Definition

A **FIFO** (named pipe) is a pipe with a name in the filesystem, which lets unrelated processes communicate. A FIFO is created with `mkfifo()` and accessed with `open()` like an ordinary file.

```
#include <sys/stat.h>
int mkfifo(const char *pathname, mode_t mode);
```

The filesystem name is only an access point. The data still lives in a kernel buffer and is never written to disk, which is why `ls -l` shows a FIFO with file type `p` and a size of 0. From the shell a FIFO is created with the `mkfifo` command.

2. Properties

A name in the filesystem. Any process that knows the path and has sufficient permissions can open the FIFO. Access is controlled by file permissions, but there is no control over exactly which process participates.

`open()` **blocks waiting for the other end.** Opening a FIFO for reading blocks until some process opens it for writing, and vice versa. A pipe has no such behaviour because `pipe()` creates both ends at once.

Two-level lifetime. The FIFO's name persists until removed with `unlink()`, while the kernel buffer exists only while some process has the FIFO open. A FIFO therefore requires manual cleanup.

Multiple readers and writers. With a pipe, multiple readers or writers appear only if the program is designed that way; with a FIFO they can appear at any time.

All pipe semantics are inherited. A FIFO keeps the byte-stream nature, EOF, partial reads and writes, blocking when the buffer is full, `SIGPIPE`, `PIPE_BUF` and `EINTR`. A FIFO only adds problems; it removes none of the pipe's. Performance is equivalent because both use the same kernel buffer and the same data path.

3. `open()` semantics

In blocking mode, both `open(O_RDONLY)` and `open(O_WRONLY)` block until the other end is opened.

In non-blocking mode the behaviour is asymmetric:

- `open(O_RDONLY | O_NONBLOCK)` always succeeds, even with no writer present.
- `open(O_WRONLY | O_NONBLOCK)` fails with `errno = ENXIO` if no reader is present.

Three error codes commonly seen when opening a FIFO mean different things: `ENOENT` means the path does not exist, `EACCES` means insufficient permissions, and `ENXIO` means the path is valid and accessible but no process is at the read end.

4. Common problems

4.1. `open()` blocks

Symptom. The reader calls `open(O_RDONLY)` and stalls until the writer calls `open(O_WRONLY)`; both calls then return at the same moment.

Code that causes it.

```
/* terminal 1, started first */
int fd = open(FIFO_PATH, O_RDONLY);   /* blocks until terminal 2 starts */


/* terminal 2, started five seconds later */
int fd = open(FIFO_PATH, O_WRONLY);   /* both calls now return */
```

Cause. `open()` on a FIFO is a **rendezvous** mechanism ensuring that two processes started at different times still meet. Without it the reader would call `read()` before any writer existed, get EOF and exit early.

Two kinds of blocking must be distinguished: `open()` blocks when no peer has opened the other end, while `read()` blocks after a successful open when the buffer is empty and a write fd still exists — that is, waiting for data.

Fix. Accept the blocking where the design allows it. To avoid it, use `O_NONBLOCK` and handle the asymmetry described in 4.2.

4.2. Asymmetric behaviour of `O_NONBLOCK`

Symptom. A non-blocking open for reading with no writer present succeeds and `read()` returns 0. A non-blocking open for writing with no reader present fails with `ENXIO`.

Code that causes it.

```
/* nobody is at the other end */
fd = open(FIFO_PATH, O_RDONLY | O_NONBLOCK);   /* succeeds */
fd = open(FIFO_PATH, O_WRONLY | O_NONBLOCK);   /* -1, errno = ENXIO */
```

Cause. Opening for reading with no writer loses no data, so it is allowed. Opening for writing with no reader means every byte written would have no recipient, so the kernel reports the error at open time.

Side effect. In non-blocking mode, `read()` returning 0 becomes ambiguous: it may mean no writer has opened the FIFO yet, or that a writer finished and closed it. Blocking mode has no such ambiguity because `open()` guarantees a writer exists before any read.

Fix. The writer handles `ENXIO` by retrying or reporting an error. The reader uses `poll()` or `select()` to wait for events on the fd instead of relying on `read()` returning 0.

4.3. The server exits after the first client

Symptom. The server reads the first client's message, `read()` returns 0 and the server exits. Later clients write into the FIFO with no recipient.

Code that causes it.

```
int read_fd = open(FIFO_PATH, O_RDONLY);


while ((n = read(read_fd, buf, sizeof(buf) - 1)) > 0)
    handle(buf, n);
/* the first client exits -> n = 0 -> loop ends, the server terminates */
```

Cause. When the client closes its write fd, the number of open write fds drops to zero and `read()` reports EOF. With a pipe the processes' lifetimes match the work, so EOF is a sensible end signal; with a FIFO the server must outlive every client, and "no client right now" does not mean "no more work".

Fix. The server opens an extra write fd to its own FIFO and holds it unused, so the write-fd count never reaches zero. The order matters: open the read end first, the write end second. Opening the write end first would block forever, because the only reader is the statement that follows.

```
int read_fd  = open(FIFO_PATH, O_RDONLY);
int dummy_fd = open(FIFO_PATH, O_WRONLY);   /* held, never used */
```

Side effect. The server no longer receives EOF, so it cannot tell when the last client is gone. If that information is needed, the server uses `poll()` to distinguish data available, `POLLHUP` (the other end closed) and timeout, or switches to a UNIX domain socket.

4.4. `SIGPIPE` when the reader exits

Symptom. The writer is killed at `write()` once the reader exits, for example when the reader is `head -3 < fifo`; the shell reports exit status 141.

Code that causes it.

```
int fd = open(FIFO_PATH, O_WRONLY);

for (int i = 1; i <= 10; i++) {
    write(fd, msg, len);   /* reader exits after 3 messages -> SIGPIPE on the 4th */
    sleep(1);
}
printf("still alive\n");   /* never runs */
```

Cause. The same mechanism as for pipes (II.5.3). The risk is higher because the reader is an independent program that may exit at any time, outside the writer's control.

Fix. Ignore `SIGPIPE` and handle `EPIPE` per file descriptor.

```
signal(SIGPIPE, SIG_IGN);

if (write(fd, buf, len) == -1 && errno == EPIPE) {
    close(fd);
}
```

4.5. Interleaved data when several clients write

Symptom. When several clients write messages larger than `PIPE_BUF` into one FIFO, their data gets interleaved.

Code that causes it.

```
/* two clients run in parallel, each writing 8000-byte messages */
memset(block, id, size);             /* id is 'A' or 'B' */
for (int i = 0; i < 20; i++)
    write(fd, block, size + 1);      /* size + 1 > PIPE_BUF -> interleaved */
```

Cause. The same mechanism as for pipes (II.5.6). The risk is higher because client code may be written by someone else; a single client writing an oversized message corrupts the data of every other client.

Fix. The protocol must require each message to be at most `PIPE_BUF` bytes and written with one `write()` call. The reader validates each frame. For large messages or many untrusted clients, use a message queue or a UNIX domain socket.

4.6. A leftover FIFO with the wrong permissions or file type

Symptom. `mkfifo(path, 0666)` fails with `EEXIST`, the program ignores the error and continues, but the FIFO's actual permissions are still the old ones (for example `0600`). A client running as a different user gets `EACCES` even though both the server and the client code are correct.

Code that causes it.

```
/* a previous run created the FIFO with mode 0600 and left it behind */
if (mkfifo(path, 0666) == -1 && errno != EEXIST) {
    perror("mkfifo");
    return 1;
}
/* EEXIST ignored and the old object reused: the mode is still 0600 */
int fd = open(path, O_RDONLY);
```

Cause. `mkfifo()` either creates a new object or fails; it never updates an existing file. A FIFO's name exists independently of any process, so a crash, a `kill -9` or a power loss leaves the name behind. Ignoring `EEXIST` is necessary for the program to be restartable, but it also makes the program accept the old object.

Related problems with the same root.

- *The path exists but is not a FIFO.* If an ordinary file occupies the path, `open()` succeeds and `write()` stores data on disk; nobody receives it and no error is reported.
- `umask` *strips permission bits.* The **umask** is the process's permission mask, which removes bits from the requested mode at creation time. With umask `022`, `mkfifo(path, 0666)` produces mode `0644`. The `mode` argument is the requested permission, not the final one.
- *The FIFO disappears after a reboot.* `/tmp` is usually cleared at boot; a long-running service should place its FIFO in `/run` or `/var/run`.

Fix. Initialize the FIFO in this order: create it with `mkfifo()` and ignore only `EEXIST`; set the exact mode with `chmod()`; open it; verify the file type with `fstat()` and the `S_ISFIFO` macro; remove the name with `unlink()` at shutdown.

The check uses `fstat()` on the open fd rather than `stat()` on the path, to avoid a **TOCTOU** (Time Of Check To Time Of Use) error, where the file changes between the check and the use.

```
if (mkfifo(path, mode) == -1 && errno != EEXIST) {
    perror("mkfifo");
    return -1;
}
if (chmod(path, mode) == -1) {
    perror("chmod");
    return -1;
```

```
}
int fd = open(path, O_RDONLY);

struct stat st;
if (fd == -1 || fstat(fd, &st) == -1 || !S_ISFIFO(st.st_mode)) {
    fprintf(stderr, "path is not a FIFO\n");
    return -1;
}
/* ... */
unlink(path);
```

4.7. Several readers split the data

Symptom. With two readers on one FIFO and a writer sending 10 messages, the total received is always 10 but the split between the readers is unpredictable. Someone running `cat` on the FIFO to inspect it also steals part of the data, with no indication to anyone.

Code that causes it.

```
/* run two copies at once, both reading the same FIFO */
int fd = open(FIFO_PATH, O_RDONLY);

while ((n = read(fd, buf, sizeof(buf) - 1)) > 0)
    count++;             /* the two readers total 10; the split is undefined */
```

Cause. A FIFO behaves as a queue: bytes taken by `read()` are removed from the buffer and no other reader sees them. When several readers wait, the kernel wakes one of them and the scheduler decides which. A FIFO does not support **broadcast** (delivering the same data to every reader), because that would require the kernel to track each reader and hold data until the slowest one has read it.

Fix. To guarantee a single reader, restrict the FIFO's permissions and place it in a private directory. For broadcast, use a separate channel per reader or a UNIX domain socket.

The same behaviour is useful for a **worker pool**: several workers read one FIFO, each task goes to exactly one worker, and the kernel distributes the work with no coordination code, provided each message is at most `PIPE_BUF` bytes.

---

5. Limits of FIFOs

A FIFO is no longer appropriate when the application needs any of the following:

- **Bidirectional communication**: a FIFO is unidirectional, so two FIFOs are required.
- **Identifying the sender**: a FIFO carries no identity of the sending process.
- **A private channel per client**: a FIFO has no demultiplexing, so one FIFO per client must be built by hand.
- **Detecting that the peer disconnected**: a FIFO has no notion of a connection.
- **Preserving message boundaries**: a FIFO is a byte stream.
- **Communication between machines**: a FIFO works within one system only.

A UNIX domain socket covers the first four, adds the notion of a connection, and exposes the peer's credentials through `SO_PEERCRED`. Message boundaries are provided by message queues or datagram sockets. Communication between machines requires an Internet socket.

IV. Shared Memory

1. Limits of the channel model

Pipes, FIFOs and message queues are **channels**: data travels from the sender, through a kernel buffer, to the receiver. Each exchange costs three things:

- **Two copies**: `write()` copies data from user space into the kernel buffer, and `read()` copies it from the kernel buffer into the receiver's user space.
- **Mode switches**: every system call switches the CPU from user mode to kernel mode and back.
- **Capacity limits**: a pipe on Linux defaults to 65536 bytes; a System V message queue on Linux defaults to at most 16384 bytes for the whole queue (`MSGMNB`) and 8192 bytes for a single message (`MSGMAX`).

With large data or a high exchange rate, the copying becomes a bottleneck. The root cause of the copying is process isolation: processes cannot see each other's memory. Shared memory removes the cost by letting several processes access the same physical memory.

2. Definition

Shared memory is the IPC mechanism that lets several processes map the same physical memory region into their own virtual address spaces. Once it is set up, the processes read and write that region directly like ordinary memory, with no system calls and no copying.

Shared memory is not a data transfer mechanism: there is no send and no receive operation. The kernel is involved only when the mapping is established and removed.

3. How it works

3.1. Page table

A **page table** is a kernel-maintained structure, one per process, mapping each virtual-address **page** onto a physical **page frame**. The **MMU** (Memory Management Unit) is the hardware unit in the CPU that uses the page table to translate virtual to physical addresses on every memory access.

Process isolation is a consequence of different processes' page tables not pointing at any common page frame. Shared memory makes entries in several processes' page tables point at the same set of page frames. The virtual address of the shared region may differ per process, but all of them map to the same physical memory.

3.2. Where the memory lives

On Linux the shared region lives in RAM with no copy on disk; the data is lost when the system shuts down. The kernel manages the region as a file on **tmpfs** — a filesystem that keeps all its contents in memory. Both System V and POSIX shared memory use this mechanism, which is why the kernel implements System V `shmat()` much like `mmap()` on a tmpfs file.

tmpfs pages can be moved to **swap** (disk storage used when RAM runs short), so shared memory is not guaranteed to stay resident in RAM. To lock the region in RAM, use `mlock()` or `shmctl(shmid, SHM_LOCK, NULL)`.

3.3. When physical memory is allocated

Linux allocates physical memory for shared memory on demand (**demand paging**):

- `shmget()`, or `shm_open()` together with `ftruncate()`, only records the object's size.
- `shmat()` or `mmap()` only creates the mapping in the address space.
- A page frame is allocated when a process first touches the page, through a **page fault** — the trap raised by the MMU when a page has no physical mapping yet.

Consequently, successful setup calls guarantee nothing about available physical memory. With POSIX shared memory, if tmpfs runs out of space when a new page is touched, the process receives `SIGBUS`.

4. When shared memory is appropriate

Shared memory fits when:

- **The data is large**: the setup cost is fixed and does not grow with the size of the data.
- **Access is repeated**: the data is read and written many times rather than sent once.

- **Access is random**: reads and writes occur at arbitrary offsets, which a byte-stream channel cannot provide.
- **Several processes read the same data**: reading does not remove it, unlike a FIFO or a message queue.

Shared memory does not fit when the data is small and exchanged rarely (a channel is simpler), when a clear message order is required (a message queue), when the receiver needs to be notified of new data and no synchronization mechanism is being built, or when the processes run on different machines.

Shared memory is the fastest IPC mechanism and the least protected: every guarantee the kernel provides in a channel is gone, and the mechanism almost always has to be paired with a synchronization mechanism.

---

5. Common problems

5.1. Race condition when updating shared data

Symptom. Several processes run `shared->counter++` on the shared region; the final value is lower than the total number of increments and varies between runs.

Code that causes it.

```
/* several processes run this on the shared region */
for (int i = 0; i < 100000; i++)
    shared->counter++;   /* load, add, store: three instructions, interleaved */
```

Cause. With no kernel involvement there is no atomicity. `counter++` takes three machine-level steps: load the value, add, store it back. When two processes interleave between those steps, one update is overwritten.

Fix. Protect the critical section with an interprocess synchronization mechanism: a POSIX named semaphore (`sem_open()`), an unnamed semaphore placed in the shared region (`sem_init()` with `pshared = 1`), or a mutex with the `PTHREAD_PROCESS_SHARED` attribute. For a simple counter, C11 atomic operations (`<stdatomic.h>`) are an option.

```
sem_t *sem = sem_open("/mysem", O_CREAT, 0600, 1);


sem_wait(sem);
shared->counter++;       /* critical section */
sem_post(sem);
```

5.2. No notification when new data arrives

Symptom. Process A finishes writing but process B has no way to know; B polls the region continuously (**busy waiting**), burning CPU.

Code that causes it.

```
/* Producer */                    /* Consumer */
write_data(shared);               while (shared->ready == 0)
shared->ready = 1;                    ;          /* busy waiting, burns CPU */
```

Cause. Shared memory has no equivalent of a blocking `read()` that wakes a process when data arrives. Writing to memory generates no event in the kernel.

Fix. Pair it with a notification mechanism: a semaphore (the writer calls `sem_post()`, the reader `sem_wait()`), a process-shared mutex and condition variable, or a side channel such as a pipe or FIFO used purely for signalling.

5.3. Observing data out of order, and what `volatile` does

Symptom. Process B sees the flag `ready = 1` while the associated data is still incomplete, even though process A wrote the data before setting the flag.

Code that causes it.

```
/* Process A */                   /* Process B */
shared->data = 42;                if (shared->ready)
shared->ready = 1;                    use(shared->data);   /* may read stale data */
```

Cause. The order of statements in the source does not determine the order other processes observe. The compiler may reorder or eliminate stores during optimization, and each core of a multi-core CPU has its own cache.

The **`volatile`** keyword only tells the compiler not to assume the variable is stable (always reload it from memory instead of keeping it in a register). `volatile` provides no atomicity, no ordering guarantee between processes, and is not a synchronization tool.

Fix. Every access to shared data, including status flags, belongs inside a critical section protected by a semaphore or a mutex. Synchronization primitives already include a **memory barrier** — the mechanism that forces the CPU and the compiler to complete stores in the intended order before a given point.

5.4. Pointers stored in the shared region are invalid

Symptom. Process A stores a pointer inside a structure in the shared region; process B reads and dereferences it and gets garbage or `SIGSEGV`.

Code that causes it.

```
/* Process A */
shared->head = alloc_in_shared_region();   /* a virtual address private to A */

/* Process B: the region is mapped at a different address */
use(shared->head->value);                  /* garbage or SIGSEGV */
```

Cause. A virtual address is meaningful only in the page table of the process that produced it. The shared region may be mapped at different virtual addresses in each process.

Fix. Store an **offset** (the distance from the start of the shared region) instead of a pointer; each process computes the real address from its own base. This applies to every linked structure: linked lists, trees, hash tables, and strings referenced by pointer.

```
b->next_off = 128;
node *n = (node *)((char *)base + b->next_off);
```

5.5. No data boundaries and undetected buffer overruns

Symptom. A 100-byte region is requested, but writing to byte 200 raises no error; only a write past the page boundary triggers `SIGSEGV`.

Code that causes it.

```
int shmid = shmget(key, 100, IPC_CREAT | 0600);   /* request 100 bytes */
char *p = shmat(shmid, NULL, 0);

p[200] = 'x';    /* still inside the 4096-byte page: no error at all */
p[5000] = 'x';   /* past the mapping: SIGSEGV */
```

Cause. The shared region is just a block of bytes with no structure and no message boundaries; any structure is imposed by the application. The kernel maps memory in whole pages, so the size is rounded up to a multiple of the page size (4096 bytes on most systems). The remainder of the last page remains accessible; access outside the mapping raises `SIGSEGV`.

Fix. Define a fixed layout for the region (a header with size, version and offsets), bounds-check every write, and obtain the page size with `sysconf(_SC_PAGESIZE)` instead of assuming it.

5.6. The object outlives the process

Symptom. The program crashes or is killed with `kill -9`; the shared memory object still occupies memory and still appears in `ipcs -m` or `/dev/shm`.

Code that causes it.

```
int shmid = shmget(key, SIZE, IPC_CREAT | 0600);
char *p = shmat(shmid, NULL, 0);

/* the process is killed with -9 before reaching this line */
shmctl(shmid, IPC_RMID, NULL);
```

Cause. Shared memory has a lifetime independent of any process (**kernel persistence**): the object exists until it is explicitly removed or the system reboots. When a process exits or calls `exec()`, the region is only unmapped from that process's address space.

- System V: `shmctl(IPC_RMID)` marks the segment for removal; it is destroyed when the last process detaches.
- POSIX: `shm_unlink()` removes the name immediately; the memory is freed when the last mapping is removed.

Fix. Call the removal function as soon as every participating process has attached, so the system destroys the object when the last process exits. Clean up manually with `ipcrm -m <shmid>` (System V) or `rm /dev/shm/<name>` (POSIX).

---

6. System V shared memory

6.1. Procedure

1. `ftok(pathname, proj_id)` generates an **IPC key** — the integer that identifies an IPC object — from an existing path and a project identifier.
2. `shmget(key, size, flags)` creates or opens the segment and returns a **shmid**, a system-wide identifier that is not a file descriptor. The size is fixed at this point.
3. `shmat(shmid, NULL, 0)` attaches the segment to the address space and returns the start address of the shared region.
4. The process accesses the memory directly, with no system calls.
5. `shmdt(addr)` detaches the segment from the address space.
6. `shmctl(shmid, IPC_RMID, NULL)` marks the segment for removal.

```
key_t key = ftok("/tmp/app", 'A');
int shmid = shmget(key, SIZE, IPC_CREAT | 0600);
```

```
char *p = shmat(shmid, NULL, 0);
/* ... */
shmdt(p);
shmctl(shmid, IPC_RMID, NULL);
```

6.2. Administration

- Read attributes with `shmctl(IPC_STAT)`; change permissions with `shmctl(IPC_SET)`.
- List with `ipcs -m`; remove manually with `ipcrm -m <shmid>`; view limits with `ipcs -l`.
- No extra linker flag is needed.

7. POSIX shared memory

7.1. Procedure

1. `shm_open(name, flags, mode)` creates or opens a named object and returns a file descriptor. The name begins with `/` (for example `/myshm`); on Linux the object appears in `/dev/shm`.
2. `ftruncate(fd, size)` sets the size, which can be changed later.
3. `mmap(NULL, size, PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0)` maps the object into the address space. The `MAP_SHARED` flag is required for changes to be visible to other processes.
4. `close(fd)`: the descriptor is no longer needed once the mapping exists, and the mapping remains valid.
5. The process accesses the memory directly.
6. `munmap(addr, size)` removes the mapping.
7. `shm_unlink(name)` removes the object's name.

```
int fd = shm_open("/myshm", O_CREAT | O_RDWR, 0600);
ftruncate(fd, SIZE);
char *p = mmap(NULL, SIZE, PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);
close(fd);
/* ... */
munmap(p, SIZE);
shm_unlink("/myshm");
```

7.2. Administration

- Read attributes with `fstat()`; change permissions with `fchmod()`.
- List with `ls -l /dev/shm`; remove manually with `rm /dev/shm/<name>`; check space with `df -h /dev/shm`.

- With glibc before 2.17 the `-lrt` flag is required; from glibc 2.17 these functions live in libc.

8. System V versus POSIX

Both APIs perform the same steps but differ in naming and operations:

- **Identifier**: System V uses a numeric IPC key produced by `ftok()`; POSIX uses a string name chosen by the application.
- **Intermediate handle**: System V returns a system-wide shmid; POSIX returns a file descriptor, so ordinary file system calls such as `fstat()` and `fchmod()` apply.
- **Size**: fixed at `shmget()` in System V; set with `ftruncate()` and changeable in POSIX.
- **Attach and detach**: `shmat()` / `shmdt()` in System V; `mmap()` / `munmap()` in POSIX.
- **Removal**: `shmctl(IPC_RMID)` in System V; `shm_unlink()` in POSIX.
- **Administration tools**: `ipcs` and `ipcrm` for System V; ordinary file operations on `/dev/shm` for POSIX.

POSIX shared memory has an API consistent with the UNIX file model and is recommended for new applications; System V remains common in older code and on systems without full POSIX IPC support.

V. Signal

1. Definition

A **signal** is an asynchronous notification sent by the kernel to a process, reporting that an event has occurred. The standard description is a **software interrupt**: the running process is suspended to handle the event and then resumes at exactly the point where it was interrupted.

A signal carries very little information — essentially a signal number, each number standing for one kind of event. Signals therefore serve for event notification and simple synchronization, not for data transfer.

2. Classification and sources

Linux defines 64 signals in two groups:

- **Standard signals**, numbers 1–31, each with a fixed meaning defined by UNIX tradition.

- **Real-time signals**, numbers 32–64, reached through `SIGRTMIN` and `SIGRTMAX`, with no predefined meaning; applications assign their own.

Some signal numbers differ between CPU architectures, so programs always use the symbolic names rather than the numbers. The full list is shown by `kill -l`.

Signals originate from four sources:

- **Hardware exceptions**: the CPU detects an invalid operation and the kernel turns it into a signal for the offending process — an invalid memory access produces `SIGSEGV`, division by zero produces `SIGFPE`.
- **Software conditions detected by the kernel**: writing to a pipe with no reader produces `SIGPIPE`, a child changing state produces `SIGCHLD` for the parent, an expired timer produces `SIGALRM`.
- **The terminal driver**: Ctrl+C produces `SIGINT`, Ctrl+\ produces `SIGQUIT`, Ctrl+Z produces `SIGTSTP`, delivered to every process in the foreground process group.
- **Another process**, through `kill()`, `raise()` or `sigqueue()`. The kernel checks permissions first: the sender must have the same real or effective user ID as the target, or be privileged.

3. The kernel structures that manage signals

The kernel maintains three separate structures per process, each holding a different kind of information.

3.1. Signal disposition

The **disposition** is the action registered for a signal. The kernel keeps an array with one entry per signal, and each entry holds:

- **A handler pointer**: the value `SIG_DFL` (default action), `SIG_IGN` (ignore), or the address of a **signal handler** — a function supplied by the program.
- `sa_mask`: the set of signals blocked while the handler runs, so the handler is not interrupted by another signal.
- `sa_flags`: behaviour flags, among them `SA_RESTART`, which asks the kernel to restart an interrupted system call, and `SA_SIGINFO`, which gives the handler extra information about the signal's origin.

The disposition is a property of the process and is shared by all its threads. It is inherited across `fork()` but reset to the default on `exec()`, since the old program's handler functions no longer exist.

Dispositions are registered with `sigaction()`. The older `signal()` function behaves differently across systems, so portable programs use `sigaction()`.

3.2. Pending signal set

Pending is the state of a signal that has been recorded but not yet acted upon. For standard signals the kernel stores this state as a **bitmask** — one bit per signal.

A consequence of the bit representation: standard signals are not queued. If the same signal arrives repeatedly while it is blocked, the bit is set only once and the process handles it only once; the remaining occurrences are lost with no notification. Real-time signals are placed in a real queue, preserving every occurrence and allowing a small value to be attached through `sigqueue()`.

3.3. Signal mask

The **signal mask** is the set of signals a process temporarily refuses to receive. A blocked signal is still recorded in the pending set and is delivered once it is unblocked.

Two states must be distinguished: a **blocked** signal is held for later delivery, while an **ignored** signal is discarded by the kernel as soon as it is generated.

The mask is changed with `sigprocmask()`, or `pthread_sigmask()` in multithreaded programs. It is a per-thread property and is inherited across both `fork()` and `exec()`.

4. Default actions and their limits

Every signal has a default action of one of five kinds:

- **Term**: terminate the process (`SIGTERM`, `SIGINT`, `SIGPIPE`).
- **Core**: terminate the process and produce a **core dump** — a memory image file used for debugging (`SIGSEGV`, `SIGABRT`, `SIGFPE`).
- **Ign**: ignore it and continue (`SIGCHLD`, `SIGWINCH`).
- **Stop**: suspend the process (`SIGSTOP`, `SIGTSTP`).
- **Cont**: resume a stopped process (`SIGCONT`).

A program may change the disposition of any signal with two exceptions: `SIGKILL` and `SIGSTOP` cannot be caught, ignored or blocked, and every attempt to change them fails. This guarantees that any process can always be terminated or stopped. In addition, `SIGCONT` always resumes a stopped process, even when it is blocked or ignored.

5. How a signal is processed

A signal's life cycle has three stages: **generated**, **pending** and **delivered**.

Generation. The kernel identifies the target process, checks the sender's permissions, and consults the disposition. If the disposition is `SIG_IGN` and the signal is neither `SIGKILL` nor `SIGSTOP`, the kernel discards the signal at once and records nothing. Otherwise it sets the corresponding bit in the pending set (or enqueues the signal, for real-time signals) and wakes the process if it is sleeping.

When the check happens. The kernel does not interrupt a process at arbitrary points. Pending signals are examined when the process transitions from kernel mode back to user mode — after a system call completes, after a timer interrupt, or when the process is rescheduled. The kernel considers the signals that are pending and not blocked; if that set is empty, the process simply continues.

Delivery. The kernel picks one signal from that set — standard signals with lower numbers first, real-time signals in queue order — clears its pending bit and acts according to the disposition.

For a default action, the kernel performs it in kernel mode: terminate the process, produce a core dump, or stop the process. The process executes none of its own instructions, which is why the system call in progress never returns and the statements after it never run.

For a handler, the kernel must run the program's own code while it is in the middle of something else, in four steps: save the current state (registers and signal mask) on the process's stack in a **signal frame**; add the signal being handled, together with `sa_mask`, to the mask to prevent recursion; switch to user mode and call the handler; and when the handler returns, a `sigreturn()` call restores the saved state and mask so the process resumes exactly where it was interrupted.

Effect on a blocked system call. If a signal arrives while the process is blocked in a system call such as `read()`, `write()` or `sem_wait()`, the call is interrupted and returns `-1` with `errno = EINTR`. If the handler was registered with the `SA_RESTART` flag, the kernel restarts the call after the handler returns instead of reporting `EINTR`.

6. The role of signals in IPC

Signals serve both communication and synchronization: a process can wait for an event with `sigsuspend()` until another process signals completion. Among the mechanisms above, `SIGPIPE` is the kernel using a signal to report a write to a channel with no receiver, and exit status 141 follows from the convention of 128 plus the signal number.

The limits are that signals carry no application data, standard signals can be lost when they arrive in bursts, and handling a signal costs more than reading a byte from a pipe. Signals therefore suit infrequent event notification and process control, not data exchange.

---

7. Common problems

7.1. Lost signals because standard signals are not queued

Symptom. The sender raises `SIGUSR1` ten times in a row; the receiver runs the handler only once or a few times, and the count varies between runs.

Code that causes it.

```
/* sender */
for (int i = 0; i < 10; i++)
    kill(pid, SIGUSR1);        /* sent in a burst */


/* receiver */
void handler(int sig) { count++; }   /* count is usually less than 10 */
```

Cause. The pending set stores standard signals as one bit each. When a signal of the same kind arrives while one is already pending, the bit is already set and the new occurrence is not recorded.

Fix. Do not use the number of deliveries as a count. A signal should only mean "there is work to do", with the amount of work kept elsewhere, such as a counter in shared memory or a queue. When every occurrence must be preserved, use a real-time signal with `sigqueue()`, keeping in mind that the queue also has a length limit.

7.2. Calling unsafe functions in a signal handler

Symptom. The program calls `printf()` or `malloc()` inside a handler; it occasionally hangs or corrupts data, and the failure does not reproduce reliably.

Code that causes it.

```
void handler(int sig) {
    printf("received signal %d\n", sig);   /* printf is not async-signal-safe */
    char *p = malloc(100);                 /* malloc is not async-signal-safe */
}
```

Cause. The handler runs in the middle of the process's own main flow. If the signal arrives while the main flow is inside `malloc()` — a moment when its internal structures are inconsistent — and the handler calls `malloc()` too, those structures are corrupted. Such a function is not **reentrant**.

POSIX defines a list of **async-signal-safe** functions that are guaranteed safe inside a handler, including `write()`, `_exit()`, `kill()` and `sigaction()`. `printf()`, `malloc()` and most of the standard library are not on that list.

Fix. Keep the handler as short as possible: set a flag and return, leaving the real work to the main flow. When a handler must print, use `write()` instead of `printf()`.

```
volatile sig_atomic_t got_signal = 0;

void handler(int sig) {
    got_signal = 1;        /* set a flag only */
}
```

7.3. Race condition when the handler touches shared data

Symptom. The handler and the main flow both read and write a global variable; the value comes out wrong, or the main flow never observes the handler's change.

Code that causes it.

```
int flag = 0;                 /* neither volatile nor sig_atomic_t */

void handler(int sig) { flag = 1; }

while (flag == 0)
    ;                         /* the compiler may keep flag in a register */
```

Cause. The handler interrupts the main flow at an arbitrary point, including between the machine instructions of a single assignment. The compiler may also keep the variable in a register instead of reloading it from memory.

Fix. Declare variables shared between a handler and the main flow as `volatile sig_atomic_t` — an integer type guaranteed to be read and written with a single machine instruction, with `volatile` forcing the compiler to reload it from memory. For more complex data, block the signal with `sigprocmask()` around the code that accesses it.

7.4. An interrupted system call returns `EINTR`

Symptom. The program is blocked in `read()` when a signal arrives; `read()` returns `-1` with `errno = EINTR` and the program treats it as a real error.

Code that causes it.

```
n = read(fd, buf, sizeof(buf));
if (n == -1) {
    perror("read");           /* wrongly reported as an error when errno = EINTR */
```

```
    return -1;
}
```

Cause. A signal delivered while the process is blocked makes the kernel end the system call early so the handler can run.

Fix. Check for `EINTR` and reissue the call, as in the `read_full()` and `write_full()` functions in section II. Alternatively, register the handler with `sigaction()` and the `SA_RESTART` flag so the kernel restarts the call. Note that `SA_RESTART` does not apply to every system call: some, including those that wait with a timeout, still return `EINTR`.

7.5. The race window between checking a flag and waiting for a signal

Symptom. The process checks a flag, sees no event, and calls `pause()` to wait; the signal arrives in the gap between the two statements and the process waits forever even though the event has already happened.

Code that causes it.

```
while (flag == 0) {            /* the signal may arrive right here */
    pause();                   /* waits forever: the signal came before pause() */
}
```

Cause. Checking the condition and entering the wait are two separate operations, and a signal can slip in between them.

Fix. Block the signal before checking the flag, then use `sigsuspend()` — a call that restores the previous mask and enters the wait as a single atomic operation, leaving no gap for a signal to slip through.

#LAB

[LAB IPC]https://github.com/trungnm2111/Session-5-Lab

Revision #12
Created 2026-09-11 07:17:25 UTC by trungnguyen
Updated 2026-09-22 02:44:50 UTC by trungnguyen